

Lazy Bureaucrat and Greedy Boss Games

Fabio Furini¹, Ivana Ljubić², Markus Sinnl³

¹PSL, Université Paris-Dauphine, France

²ESSEC Business School of Paris, France

³Department of Statistics and Operations Research, University of Vienna, Austria

Abstract

Given a set of items with profits and weights and a knapsack capacity, we study the problem of finding a maximal knapsack packing that minimizes the profit of selected items. We propose an effective dynamic programming (DP) algorithm which has pseudo-polynomial time complexity. We demonstrate the equivalence between this problem and the problem of finding a minimal knapsack cover that maximizes the profit of selected items. In an extensive computational study on a large and diverse set of benchmark instances, we demonstrate that the new DP algorithm outperforms a state-of-the-art commercial mixed-integer programming (MIP) solver applied to the two best performing MIP models from the literature.

Keywords: combinatorial optimization, maximal knapsack packing, minimal knapsack cover, dynamic programming, integer programming

1. Introduction

One of the most famous and most studied problems in combinatorial optimization is the classical *Knapsack Problem* (KP) which is defined as follows: Given an knapsack capacity $C > 0$ and a set $I = \{1, \dots, n\}$ of items, with profits $p_i \geq 0$ and weights $w_i \geq 0$, KP asks for a maximum profit subset of items whose total weight does not exceed the capacity. The problem can be formulated using the following mixed-integer program (MIP):

$$\max \left\{ \sum_{i \in I} p_i x_i : \sum_{i \in I} w_i x_i \leq C, x_i \in \{0, 1\}, i \in I \right\} \quad (1)$$

where each variable x_i takes value 1 if and only if item i is inserted in the knapsack. Without loss of generality we can assume that all input parameters have integer values. In the following, we will refer to (I, p, w, C) as a *knapsack instance*. KP is NP-hard, but it is well-known that fairly large instances can be solved to optimality within short computing times (see, e.g., [14, 15, 17] for comprehensive surveys on applications and variants of KP).

Optimization problems related to KP search for solutions satisfying one of the following two properties:

Definition 1.1 (Maximal Knapsack Packing). Given a knapsack instance (I, p, w, C) , a packing $\mathcal{P} \subset I$ is a subset of items which does not exceed the capacity C , i.e., $\sum_{i \in \mathcal{P}} w_i \leq C$. A packing $\mathcal{P}$ is called *maximal* if and only if $\mathcal{P} \cup \{i\}$ is not a packing for any $i \notin \mathcal{P}$.

Definition 1.2 (Minimal Knapsack Cover). Given a knapsack instance (I, p, w, C) , a cover $\mathcal{K} \subset I$ is a subset of items which exceeds or it is equal to the capacity C , i.e., $\sum_{i \in \mathcal{K}} w_i \geq C$. A cover $\mathcal{K}$ is called *minimal* if and only if $\mathcal{K} \setminus \{i\}$ is not a cover for any $i \in \mathcal{K}$.

In this article we study weighted versions of the *knapsack covering* and *knapsack packing* problems which are defined below.

Definition 1.3 (Minimum-Cost Maximal Knapsack Packing (MCMKP)). Given a knapsack instance (I, p, w, C) , the goal of the MCMKP is to find a maximal knapsack packing $S^* \subset I$ that minimizes the profit of selected items, i.e.:

$$S^* = \arg \min_{S \subset I} \left\{ \sum_{i \in S} p_i \mid \sum_{i \in S} w_i \leq C \text{ and } \sum_{i \in S} w_i + w_j > C, \forall j \notin S \right\}.$$

Definition 1.4 (Maximum-Profit Minimal Knapsack Cover (MPMKC)). Given a knapsack instance (I, p, w, C) , the goal of the MPMKC is to find a minimal knapsack cover $S^* \subseteq I$ that maximizes the profit, i.e.:

$$S^* = \arg \max_{S \subset I} \left\{ \sum_{i \in S} p_i \mid \sum_{i \in S} w_i \geq C \text{ and } \sum_{i \in S \setminus \{j\}} w_i < C, \forall j \in S \right\}.$$

In both MCMKP and MPMKC, we assume that the problem instance (I, p, w, C) is non-trivial, i.e., the corresponding optimal solution S^* is a proper subset of I .

In the classical KP, one tries to pack items while *maximizing* their (non-negative) profit, hence, every optimal KP solution trivially satisfies the maximal knapsack packing property. On the contrary, the objective function of the MCMKP goes in the opposite direction – one searches for a subset of items to pack, while *minimizing* the profit of selected items. Hence, solving the standard KP in which the profit-maximization objective is replaced by profit-minimization results in a trivial (empty) solution. This is why the maximal packing property has to be explicitly imposed when searching for an optimal MCMKP solution.

The MCMKP models applications related to scheduling jobs on a single machine with common arrival times and a common deadline. The problem consists of selecting a subset of jobs to schedule (before the given deadline) and accordingly the order in which the jobs are scheduled becomes irrelevant. The set I refers to all possible jobs, job durations are given as w_i and p_i is the cost for executing job i , $i \in I$. The common deadline C corresponds to the capacity of the knapsack. The goal is to minimize the cost for executing scheduled jobs while at the same time ensuring that the maximal subset of jobs is scheduled (i.e., a maximal packing is preserved). Consequently, the scheduling task boils down to solving the MCMKP. The problem has been

called the Lazy Bureaucrat Problem (LBP) with common arrivals and deadlines (see, e.g., [11]), motivated by the special case in which $p_i = w_i$, for all $i \in I$. We deliberately refrain from using this name since in this article we study a more general setting in which no correlation is assumed to exist between the item profits and item weights.

Similarly, in case of MPMKC, the goal is to maximize the profit of scheduled jobs while ensuring that a minimal possible subset of jobs exceeding the deadline is scheduled (i.e., to preserve a minimal cover). The MPMKC problem has been called the Greedy Boss Problem in [11], but again, since this name is also motivated by the setting $p_i = w_i$, for all $i \in I$, we will not use it in the remainder of this article.

Literature Overview The MCMKP problem has been introduced in [1] where it was shown that a more general problem variant with individual arrival times and deadlines is NP-hard. Following this seminal work, the problem has been extensively studied in the Computer Science literature and results concerning the problem complexity of special classes and approximation algorithms have been presented ([8, 11, 13]). On the contrary, concerning the development of exact solution methods, the problem has been tackled only in a recent preliminary study [7].

Depending on the coefficients p_i of the objective function, the following special cases are known in the (scheduling) literature:

- **min-number-of-jobs**: In this case, $p_i = 1$ for all $i \in I$ and the goal is to minimize the number of scheduled jobs. By reduction from the subset-sum problem, it has been shown in [8] that this problem variant is weakly NP-hard.
- **min-time-spent**: This variant corresponds to the problem in which $p_i = w_i$ for all $i \in I$. Since job profits are equal to job durations, this explains the motivation for calling the problem LBP, i.e., the goal of the “lazy employee” is to go home as early as possible, while having an excuse that there is no more time left to start a new job. This variant is also equivalent to the **min-makespan**, since all jobs arrive at the same time, and the goal is to minimize the time spent on executing jobs. In [11] two greedy heuristics and an FPTAS have been proposed for the **min-time-spent** objective with common arrivals and deadlines.
- **min-weighted-sum**: This variant considers the most general objective function, with no restrictions on p_i values, and hence it covers all previous cases. This is the objective function considered in our paper. In [7], an $O(n^2C)$ dynamic programming procedure is proposed, thus showing that MCMKP is weakly NP-hard in this general setting. In addition, three MIP formulations and their constraint programming (CP) counterparts are developed and tested, showing that the best performance is obtained by two of the proposed MIP models (these models are revised in Section 4).

Our Contribution This article aims at providing an in-depth study of exact methods (based on MIP and DP techniques) for solving MCMKP and MPMKC to provable optimality. Our contribution is as follows: First, we provide a dynamic programming algorithm for the MCMKP

that runs in $O(nC)$ time and $O(n + C)$ space. It improves upon the previous DP from [7] which was shown to run in $O(n^2C)$ time and $O(nC)$ space. Second, we compare the quality of the linear programming (LP) relaxations of the two best performing MIP models from the literature. In an extensive computational study on a large and diverse set of benchmark instances, we demonstrate that the new DP algorithm outperforms a state-of-the-art commercial MIP solver applied to these two best performing models.

Finally, we prove that MCMKP and MPMKC are equivalent and that any optimal solution of the MPMKC can be obtained by an appropriate linear transformation. This implies that MPMKC is weakly NP-hard as well, and that there exists a DP procedure which runs in $O(n(W - C))$ time and requires $O(n + (W - C))$ space, where W is the sum of all item weights.

The remainder of this article is organized as follows: in the following section, we present structural properties of optimal MCMKP solutions that can be used to derive DP algorithms and MIP models. Section 3 is devoted to our new DP procedure for the MCMKP. In Section 4, we present and discuss new results for two MIP models for the MCMKP. In Section 5 we prove the equivalence between the MCMKP and the MPMKC. Computational results are provided in Section 6 and conclusions are drawn in Section 7.

2. Solution Properties for the MCMKP

In this section we point out some structural solution properties of MCMKP solutions that will be used in the remainder of this article.

In the following, we assume that all items are sorted in non-decreasing order according to their weight, i.e., $w_1 \leq w_2 \leq \dots \leq w_n$. For each $i \in I$, let $C_i := C - w_i$, so we have $C_1 \geq C_2 \geq \dots \geq C_n$. We will also denote by $w_{\max} := \max_{i \in I} w_i (= w_n)$. Let $W := \sum_{i \in I} w_i$ and $P := \sum_{i \in I} p_i$. For a set $S \subset I$, let $w(S) = \sum_{i \in S} w_i$ and let $W_i = \sum_{j < i} w_j$.

It is not difficult to see that the following property is valid, due to the maximal packing property of an arbitrary MCMKP solution.

Property 2.1 ([7]). *The capacity used by an arbitrary feasible MCMKP solution $S \subset I$ is bounded from below as follows:*

$$w(S) \geq C - w_{\max} + 1.$$

In the following, we define a *critical item*, as an item whose weight gives a tight upper bound on the smallest possible item left out of any feasible solution.

Definition 2.2 (MCMKP Critical Weight and MCMKP Critical Item). Denote by

$$i_c = \arg \min \left\{ i \in I \mid \sum_{j \leq i} w_j > C \right\}$$

the index of the *critical item*, i.e., the index of the first item that exceeds the capacity, assuming all $i \leq i_c$ will be taken as well. The *critical weight*, denoted by w_c , is the weight of the critical item, i.e., $w_c = w_{i_c}$.

Proposition 2.3 ([7]). *The weight of the smallest item left out of any feasible MCMKP solution S is bounded from above by the critical weight w_c , i.e.:*

$$S \text{ is feasible} \Rightarrow \min_{i \notin S} w_i \leq w_c.$$

Consequently, the size of the knapsack can be bounded from below as:

$$w(S) \geq C - w_c + 1.$$

The validity of the above proposition follows from the simple observation: assume that there exists a feasible knapsack packing $S \subset I$ such that the weight of the smallest item left outside of S is $> w_c$. In that case, S must contain all items i such that $w_i \leq w_c$. By definition of w_c , the knapsack capacity occupied by S is then strictly greater than C , which is a contradiction.

Proposition 2.4. *Let $S \subset I$ be a feasible MCMKP solution. If item $i \in I$ is the smallest item left out of S , i.e., we have $S = \{1, \dots, i-1\} \cup S'$ where $S' \subseteq \{i+1, \dots, n\}$, the weight of S' can be bounded from below as:*

$$w(S') \geq C - \sum_{j < i} w_j - w_i + 1.$$

Proof. Assume that $w(S') \leq C - \sum_{j < i} w_j$. In that case, item i could be added to S without exceeding the knapsack capacity (as $w(S') \geq 0$ and the sum of the right-hand side includes w_i), which contradicts the assumption that the packing S is maximal. $\square$

Property 2.5. *Despite the minimization objective, imposing the upper bound on the size of the knapsack is not redundant, in general.*

In order to see that the latter property holds, consider the following example in which we are given three items such that $w_1 = 1$, $w_2 = 2$ and $w_3 = 3$, and $p_1 = 10$, $p_2 = p_3 = 1$, and let $C = 4$ be the knapsack capacity. Without imposing the upper bound on the size of the knapsack, the optimal solution will be to take the items $\{2, 3\}$ with the total cost of 2, whereas the optimal MCMKP solution is $\{1, 3\}$ with the total cost of 11.

In the **min-time-spent** case, however, imposing the upper bound is indeed redundant, as the next proposition shows.

Proposition 2.6. *If $w_i = p_i$ for all $i \in I$, the capacity of any optimal solution S will not exceed C , even without explicitly imposing this condition.*

Proof. Let us denote the MCMKP without the knapsack capacity constraint by *relaxed problem*. Assume that $w_i = p_i$, for each $i \in I$, and that, on the contrary, S is an optimal solution (of

minimal cardinality) of the relaxed problem, such that $w(S) > C$. Let k be the index of the least weight item in S , i.e., $k = \arg \min_{i \in S} w_i$. We will construct another solution S' as $S' = S \setminus \{k\}$. We have $w(S') + w_i \leq C$ for some item $i \in I \setminus S'$, since otherwise S' would be a maximal knapsack packing with a better objective value than S , which is a contradiction.

Let $\delta = C - w(S')$, which is the residual capacity of the knapsack that we obtain by removing the item k from S . We will now try to fill this gap by solving the following subproblem:

$$\max \sum_{i \in I \setminus S'} w_i x_i \quad \text{s.t.} \quad \sum_{i \in I \setminus S'} w_i x_i \leq \delta, \quad x_i \in \{0, 1\}.$$

Denote by S'' the optimal solution to this problem. Clearly $S'' \neq \emptyset$. Then, the new solution $\tilde{S} = S' \cup S''$ satisfies the following property: $w(\tilde{S}) \leq C$ and (by construction) $w(\tilde{S}) + w_i > C$ for all $i \notin \tilde{S}$, i.e., $\tilde{S}$ is another feasible solution of the relaxed problem with a better objective value than S , a contradiction. $\square$

3. A New $O(nC)$ Dynamic Programming Algorithm for Solving the MCMKP

In this section we derive a new dynamic programming (DP) algorithm for the MCMKP. The DP is based on the fact that, once the smallest item left out of the solution is known, MCMKP reduces to solving the knapsack problem with a lower and upper bound on its capacity (denoted LU-KP in the following). This property can be stated as follows:

Proposition 3.1. *The optimal MCMKP solution can be calculated as follows:*

$$\text{OPT} = \min_{i \in I: i \leq i_c} \left\{ KP(i, \underline{C}, \overline{C}) + \sum_{j < i} p_j \right\} \quad (2)$$

where $\underline{C} = C - W_i - w_i + 1$ and $\overline{C} = C - W_i$, and

$$KP(i, \underline{C}, \overline{C}) = \min \left\{ \sum_{j > i} p_j y_j : \underline{C} \leq \sum_{j > i} w_j y_j \leq \overline{C}, y \text{ is binary} \right\}. \quad (3)$$

Proof. Let $\tilde{X}$ be an optimal MCMKP solution. Let item i be the smallest item taken out of $\tilde{X}$. Hence, all items from the set $\{1, \dots, i-1\}$ belong to $\tilde{X}$. These items can be excluded from consideration, reducing the maximum knapsack capacity from C to $\overline{C} = C - W_i$. Then, the value of $\tilde{X}$ can be obtained as $KP(i, \underline{C}, \overline{C}) + \sum_{j < i} p_j$, where the LU-KP problem defined by (3) is solved on the remaining items from the set $\{i+1, \dots, n\}$. Since solution $\tilde{X}$ has to be a maximal packing, the minimum capacity is defined as $\underline{C} = C - W_i - w_i + 1$ (see Property 2.4).

The optimal MCMKP solution value is found by trying all possible items $i \leq i_c$ and taking the best among the obtained solutions. $\square$

We now introduce the new DP algorithm which runs in $O(nC)$ time, and requires only $O(n + C)$ space. The pseudo-code of this new DP procedure is given in Algorithm 1. A single (look-up) vector M of size C is used to keep track of the solution value for the problem $KP(i, 0, C)$. The update of values of M follows the DP procedure for solving a knapsack problem in which one searches for the subset of items whose total weight is exactly k ($k \in \{0, 1, \dots, C\}$) such that the sum of profits of selected items is *minimized*. Accordingly, at each stage i ($i = n, \dots, 1$), the value $M[k]$ stores the minimum profit of the subset of items that occupies exactly capacity k , while considering only the items from $\{i + 1, \dots, n\}$. Once the stage i is reached, such that $i \leq i_c$ (i.e., all items not larger than i can fit into the knapsack), the algorithm updates the best solution value according to formula (2).

Algorithm 1: An $O(nC)$ dynamic programming framework.

Data: Set of items I with profits p_i and weights w_i , $i \in I$. Knapsack capacity C .

Sums of profits $P_i = \sum_{j < i} p_j$ and weights $W_i := \sum_{j < i} w_j$;

Result: The optimal MCMKP solution value OPT, required capacity C^* and the index i^* of the smallest item left outside of the solution.

```

1 Set  $M[0] := 0$ ,  $\text{OPT} := \infty$ ;
2 for  $k = 1, \dots, C$  do
3   | Set  $M[k] := \infty$ 
4 end
5 for  $i = n, \dots, 1$  do
6   | if  $i \leq i_c$  then
7     | Set  $\bar{C} := \max\{0, C - W_i\}$ , and  $\underline{C} = \max\{0, C - W_i - w_i + 1\}$ ;
8     | Set  $\text{tmp} := \min_{\underline{C} \leq k \leq \bar{C}} \{M[k] + P_i\}$ ;
9     | if  $\text{tmp} < \text{OPT}$  then
10    | | Set  $\text{OPT} := \text{tmp}$ ,  $i^* := i$ ,  $C^* := \arg \min_{\underline{C} \leq k \leq \bar{C}} \{M[k] + P_{i^*}\} + W_{i^*}$ ,
11    | end
12   | end
13   | for  $k = C, \dots, w_i$  do
14     | | Set  $M[k] := \min\{M[k], M[k - w_i] + p_i\}$ ;
15   | end
16 end
17 return  $\text{OPT}, C^*, i^*$ ;
```

During the execution of the algorithm a look-up table of size $O(nC)$ can also be built, with entries $M_i[k]$ storing the best LU-KP solution value considering items $\{i + 1, \dots, n\}$ and capacity k , $k \in \{0, \dots, C\}$. A small example that illustrates our DP procedure (with the entries of matrix M) is given in Table 1.

Proposition 3.2. *The optimal solution value of the MCMKP can be computed by Algorithm 1 in $O(nC)$ time.*

Proof. The key element of this new DP algorithm is the fact that the DP for the classical KP (see, e.g., [3, 15]) provides all necessary information to compute the optimal MCMKP solution value. While processing an item i (note that the items are processed in the backward direction, from the largest to the smallest one), we can compute the optimal values of the LU-KP considering

item			Capacity										
i	p_i	w_i	0	1	2	3	4	5	$\underline{C}$	$\overline{C}$	P_i	W_i	OPT
5	1	4	0	∞	∞	∞	1	∞	-	-	15	7	-
4	2	3	0	∞	∞	2	1	∞	0	1	13	4	13
3	6	2	0	∞	6	2	1	8	2	3	7	2	9
2	4	1	0	4	6	2	1	5	4	4	3	1	4
1	3	1	0	3	6	2	1	4	5	5	0	0	4

Table 1: Let the input instance be given as $|I| = 5$ with $w = (1, 1, 2, 3, 4)$ and $p = (3, 4, 6, 2, 1)$ and capacity $C = 5$. The index of the critical item is $i_c = 4$. The table represents the content of the vector M in the stages $i = 5, \dots, 1$. The optimal solution is $S^* = \{1, 5\}$, $C^* = 5$ and $i^* = 2$.

an instance restricted to the subset of items $\{i + 1, \dots, n\}$ and for all $k \in \{0, \dots, C\}$. This is done by implementing the following recursive procedure (cf. Line 14 in Algorithm 1):

$$M_i[k] := \begin{cases} \min\{M_i[k], M_{i+1}[k - w_i] + p_i\}, & \text{if } k \geq w_i \\ M_{i+1}[k], & \text{otherwise} \end{cases} \quad \forall i = n, \dots, 1, k = 0, \dots, C$$

where the initial values $M_{n+1}[k] := \infty$, for all $k = 1, \dots, C$ and $M_{n+1}[0] := 0$.

Exploiting the relation between the MCMKP and the LU-KP solution values (cf. Proposition 3.2), in each iteration i (starting from an item i such that $i \leq i_c$) we then obtain the optimal MCMKP solution value, assuming all unprocessed items (i.e., those from the set $\{1, \dots, i - 1\}$) are taken in the solution. The value of this solution is stored in the variable tmp (cf. Line 8), and the globally optimal solution value is updated correspondingly. Observe that the items $\{1, \dots, i - 1\}$ determine the remaining capacity of the knapsack (which is given as $\overline{C}$) that has to be filled using items from $\{i + 1, \dots, n\}$, whereas $\underline{C}$ is determined based on the fact that the packing has to be maximal and that the item i is the smallest one taken out of the solution (cf. Property 2.4). This concludes the proof. $\square$

Observe that, so far, Algorithm 1 only returns the optimal solution value OPT. In order to recover the optimal solution S^* , two possibilities exist. At the cost of increasing the space complexity from $O(n + C)$ to $O(nC)$, the optimal solution can be recovered in $O(n)$ time. To this end, it is sufficient to store the information whether the item i was added to the LU-KP knapsack in the current iteration or not, for each weight $k \in \{0, \dots, C\}$ and in each iteration $i = n, \dots, 1$. We keep track of the pointers $A_i[k] \in \{0, 1\}$ with the following meaning:

$$A_i[k] := \begin{cases} 1, & \text{if } M_i[k] = M_{i+1}[k - w_i] + p_i \\ 0, & \text{otherwise (i.e., if } M_i[k] = M_{i+1}[k]) \end{cases}, \quad \forall i = n, \dots, 1, \quad k = 0, \dots, C$$

After obtaining the optimal solution value OPT, we can compute the optimal MCMKP solution S^* by applying a slight modification of the backtracking procedure for the standard KP (for further details concerning this backtracking see, e.g. Section 2.3 in [14]). In the following, we explain necessary modifications to this procedure. Recall that Algorithm 1 returns the capacity C^* required by the optimal solution. We first initialize $S^* = \{1, \dots, i^* - 1\}$, and then, starting

from $A_{i^*}[C^* - W_{i^*}]$, we complete S^* following the backtracking procedure for KP.

In the following we show that the optimal solution can be recovered, while keeping the space complexity $O(n + C)$.

Proposition 3.3. *The optimal solution and the optimal solution value of the MCMKP can be computed in $O(nC)$ time and $O(n + C)$ space.*

Proof. We observe that, once Algorithm 1 returns the index i^* of the smallest item left out from the optimal solution, the MCMKP boils down to solving the LU-KP with $\underline{C} = \overline{C} = C^* - W_{i^*}$ and over the set of items $I' = I \setminus \{1, \dots, i^*\}$. Observe that, when $\underline{C} = \overline{C}$, the LU-KP is also known as the *Equality Knapsack Problem* (EKP) [14] (in a minimization form). Hence, to recover the optimal solution, it is sufficient to recover the optimal solution of the EKP and take the union with the set of items $I'' = \{1, \dots, i^* - 1\}$. Notice that equivalent solutions produced by the EKP may only differ on the set of items I' . After the union with I'' , each of them is feasible for the MCMKP and, since they have the same objective function value, they are all optimal.

For recovering an optimal EKP solution, we use the recursive procedure described in Section 3.3 of [14]. Following the notation of [14], we are given a set N of input elements (corresponding to the items) and an integer parameter $\mathcal{C}$ (corresponding to the capacity that should be satisfied with equality). Let $X^*(N, \mathcal{C})$ be an optimal solution set and $z^*(N, \mathcal{C})$ be the optimal solution value. For the sake of completeness we repeat the recursive algorithm from [14]. The algorithm uses three global variables N^* , $\mathcal{C}^*$ and $X^*(N^*, \mathcal{C}^*)$ storing the currently processed set of elements, the capacity used by the current solution and the solution itself, respectively. Algorithm 2 is started by setting $N^* := \emptyset$ and $\mathcal{C}^* := 0$ and performing **Recursive-DP**($N, \mathcal{C}, z^*(N, \mathcal{C})$). At the end of the recursion, the optimal EKP solution set can be finally found as $X^*(N, \mathcal{C}) := X^*(N^*, \mathcal{C}^*)$.

An optimal MCMKP solution is calculated as $I'' \cup X^*(N, \mathcal{C})$ where $X^*(N, \mathcal{C})$ is obtained by running the procedure **Recursive-DP** with $N = I'$, $\mathcal{C} = C^* - W_{i^*}$ and $z^*(N, \mathcal{C}) = \text{OPT} - P_{i^*}$ and with the procedure **Solve**($N, \mathcal{C}$) that solves the EKP.

As proven in [14], Algorithm 2 runs in $O(|N|\mathcal{C})$ time, requires $O(|N| + \mathcal{C})$ space and it computes an optimal solution if the following four properties hold:

1. There exists a procedure **Solve**($N, \mathcal{C}$) which computes for every integer $k = 0, \dots, \mathcal{C}$ the optimal value $z^*(N, k)$ and requires $O(|N|\mathcal{C})$ time and $O(|N| + \mathcal{C})$ space.
2. If $|N| = 1$, then **Solve**($N, \mathcal{C}$) also computes the optimal solution $X^*(N, k)$ for each $k = 0, \dots, \mathcal{C}$.
3. For every partition N_1, N_2 of N , there exist capacities $\mathcal{C}_1, \mathcal{C}_2$ such that $\mathcal{C}_1 + \mathcal{C}_2 = \mathcal{C}$ and $z^*(N_1, \mathcal{C}_1) + z^*(N_2, \mathcal{C}_2) = z^*(N, \mathcal{C})$.
4. For every partitioning N_1, N_2 of N with $|N_2| = 1$, and arbitrary capacities $\mathcal{C}_1, \mathcal{C}_2$, there exists a procedure **Merge** that combines $X^*(N_1, \mathcal{C}_1)$ and $X^*(N_2, \mathcal{C}_2)$ into $X^*(N_1 \cup N_2, \mathcal{C}_1 + \mathcal{C}_2)$ in $O(\mathcal{C}_2)$ time.

Algorithm 2: Recursive-DP($N, \mathcal{C}, z^*(N, \mathcal{C})$).

Result: $X^*(N, \mathcal{C})$.

```
/* global variables:   $N^*$ ,  $\mathcal{C}^*$  and  $X^*(N^*, \mathcal{C}^*)$  */
/* Divide: */
1 Partition  $N$  into two disjoint subsets  $N_1$  and  $N_2$  of approximately the same size ;
2 Perform Solve( $N_1, \mathcal{C}$ ) computing  $z^*(N_1, k)$  for  $k = 0, \dots, \mathcal{C}$  ;
3 Perform Solve( $N_2, \mathcal{C}$ ) computing  $z^*(N_2, k)$  for  $k = 0, \dots, \mathcal{C}$  ;
4 Find  $\mathcal{C}_1, \mathcal{C}_2$  with  $\mathcal{C}_1 + \mathcal{C}_2 = \mathcal{C}$  and  $z^*(N_1, \mathcal{C}_1) + z^*(N_2, \mathcal{C}_2) = z^*(N, \mathcal{C})$  ;
/* Conquer: */
5 if  $|N_1| = 1$  then
6   Merge  $X^*(N_1, \mathcal{C}_1)$  and  $X^*(N^*, \mathcal{C}^*)$  into  $X^*(N_1 \cup N^*, \mathcal{C}_1 + \mathcal{C}^*)$ ;
7   Set  $N^* := N_1 \cup N^*, \mathcal{C}^* := \mathcal{C}_1 + \mathcal{C}^*$ 
8 end
9 if  $|N_2| = 1$  then
10  Merge  $X^*(N_2, \mathcal{C}_2)$  and  $X^*(N^*, \mathcal{C}^*)$  into  $X^*(N_2 \cup N^*, \mathcal{C}_2 + \mathcal{C}^*)$ ;
11  Set  $N^* := N_2 \cup N^*, \mathcal{C}^* := \mathcal{C}_2 + \mathcal{C}^*$ 
12 end
/* Recursion: */
13 if  $|N_1| > 1$  then Recursive-DP( $N_1, \mathcal{C}_1, z^*(N_1, \mathcal{C}_1)$ );
14 if  $|N_2| > 1$  then Recursive-DP( $N_2, \mathcal{C}_2, z^*(N_2, \mathcal{C}_2)$ );
```

Regarding the first two properties, Algorithm 3 provides a pseudo-code for the Solve($N, \mathcal{C}$) procedure that finds the optimal EKP solution value and returns the optimal solution $X^*(N, k)$ if N is a singleton, for each $k = 0, \dots, \mathcal{C}$. In the same way as for the KP, the EKP satisfies the third property due to the linearity of the problem and the Merge procedure required by the fourth property is a set union procedure.

Algorithm 3: Solve($N, \mathcal{C}$).

Data: Set of items N with profits p_i and weights w_i , $i \in N$. Knapsack capacity $\mathcal{C}$. ;**Result:** Opt. EKP solution values $z^*(N, k)$ and $X^*(N, k)$ (if $|N| = 1$) for all $k = 0, \dots, \mathcal{C}$

```
1 Set  $z^*(N, 0) := 0$  ;
2 for  $k = 1, \dots, \mathcal{C}$  do
3   Set  $z^*(N, k) := \infty$ ;
4   if  $|N| = 1$  then  $X^*(N, k) = \emptyset$  ;
5 end
6 if  $|N| = 1$  then  $X^*(N, 0) = \emptyset, X^*(N, w_{|N|}) = N$ ;
7 for  $i = |N|, \dots, 1$  do
8   for  $k = \mathcal{C}, \dots, w_i$  do
9     Set  $z^*(N, k) := \min\{z^*(N, k), z^*(N, k - w_i) + p_i\}$ 
10  end
11 end
```

□

4. MIP Models for the MCMKP

Several MIP and constraint programming (CP) formulations for the MCMKP have been proposed in [7] where it has been demonstrated that CP formulations are not competitive against MIP models. In the following, we present the two best performing MIP models, both of which use binary variables x_i , for all $i \in I$, representing the selected items in the knapsack. The first formulation is in the natural variable space, and the second one in an extended space. We then derive new interesting connections between the two models, comparing the strength of their LP relaxations.

4.1 A Lifted MIP Formulation for the MCMKP in the Natural Variable Space

The first formulation, denoted by (F_l) , reads as follows:

$$(F_l) \quad \min \sum_{i \in I} p_i x_i \quad (4)$$

$$\sum_{i \in I} w_i x_i \leq C \quad (5)$$

$$\sum_{j \in I, j \neq i} \min\{w_j, C_i + 1\} x_j + \min\{w_c, C_i + 1\} x_i \geq C_i + 1 \quad \forall i \in I : i \leq i_c \quad (6)$$

$$x_i \in \{0, 1\} \quad \forall i \in I \quad (7)$$

Constraint (5) is a knapsack constraint stating that the weight of all selected items cannot exceed the available capacity C . As shown in [7], inequalities (6) are obtained by down lifting the inequalities:

$$\sum_{j \in I, j \neq i} w_j x_j + (C_i + 1) x_i \geq C_i + 1 \quad \forall i \in I : i \leq i_c \quad (8)$$

which are based on Proposition 2.3 and Property 2.4. The latter inequalities ensure the maximal packing property, i.e., they make sure that adding each additional item i such that $i \leq i_c$ will exceed the total capacity.

4.2 An Extended Formulation

The second formulation, denoted by (F_e) , requires an additional non-negative continuous variable z and it reads as follows:

$$\begin{aligned}
(F_e) \quad & \min \sum_{i \in I} p_i x_i \\
& \sum_{i \in I} w_i x_i \leq C
\end{aligned} \tag{9}$$

$$\sum_{i \in I} w_i x_i + z \geq C + 1 \tag{10}$$

$$z \leq w_c - (w_c - w_i)(1 - x_i) \quad \forall i \in I : i \leq i_c \tag{11}$$

$$x_i \in \{0, 1\} \quad \forall i \in I \tag{12}$$

$$z \geq 0 \tag{13}$$

Due to the coupling constraints (11), the value of the variable z is bounded from above by the weight of a smallest item that is left out of the solution. The maximal packing property of any feasible solution is ensured by the covering constraint (10). Due to the Proposition 2.3, it is not necessary to impose constraints (11) for $i > i_c$.

Compared to the (F_l) model, the (F_e) model has one additional variable but the structure of the constraint matrix is significantly simplified. Besides the packing constraint (9) and the covering constraint (10), the remaining matrix has a diagonal structure, plus one column of ones (corresponding to the variable z).

In the following, using Fourier-Motzkin elimination (see, e.g., [21]), we show that this model leads to an alternative formulation in the natural space of variables.

Proposition 4.1. *Consider the following model denoted by (F_z) :*

$$\begin{aligned}
(F_z) \quad & \min \sum_{i \in I} p_i x_i \\
& \sum_{i \in I} w_i x_i \leq C
\end{aligned} \tag{14}$$

$$\sum_{j \in I, j \neq i} w_j x_j + w_c x_i \geq C_i + 1 \quad \forall i \in I : i \leq i_c \tag{15}$$

$$x_i \in \{0, 1\} \quad \forall i \in I \tag{16}$$

The value of the LP relaxations of the models (F_e) and (F_z) are the same.

Proof. Formulation (F_z) obviously models the problem, as constraints (15) are a down-lifted variant of constraints (8). To show that the value of the LP relaxations are the same, we eliminate the variable z from the LP relaxation of (F_e) by combining the following inequalities:

$$\begin{aligned}
C + 1 - \sum_{i \in I} w_i x_i & \leq z \\
z & \leq w_c - (w_c - w_i)(1 - x_i) \quad \forall i \in I : i \leq i_c.
\end{aligned}$$

After rewriting $C + 1 - \sum_{i \in I} w_i x_i \leq w_c - (w_c - w_i)(1 - x_i)$, we obtain inequalities (15). Since the variable z does not appear in the objective function, the value of the LP relaxation does not change. $\square$

Observe that the only difference between constraints (15) and (8) is in the coefficient next to the variable x_i , and that by standard arguments based on the value of the right-hand side, constraints (15) can be down-lifted to (6). So, depending on the item weights of the input instance, the formulations (F_l) and (F_e) do not necessarily provide the same value of the LP relaxation. More precisely, we have:

Corollary 4.2. *The LP relaxation bounds of (F_l) are never worse than the LP relaxation bounds of (F_e) , and **there exist instances for which** the LP relaxation bounds of (F_l) are strictly better.*

In order to see that, in general, the formulation (F_l) can be strictly stronger than (F_e) , consider the same example given in Table 1. Inequalities (6) and (15) are identical for all $i < i_c$, however for $i = i_c = 4$, the inequality (6) in the model (F_l) is $x_1 + x_2 + 2x_3 + 3x_4 + 3x_5 \geq 3$, whereas the associated inequality (15) is weaker and reads $x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 \geq 3$.

However, if the weight of the critical item is "relatively small" compared to the overall knapsack capacity, then the two models provide the same LP relaxation. We have:

Corollary 4.3. *If $w_{\max} + w_c \leq C + 1$, then the LP relaxation bounds of (F_l) and (F_e) are the same.*

Proof. Since $w_{\max} + w_c \leq C + 1$, we have that $w_j + w_c \leq C + 1$ (and hence $w_j \leq C_c + 1$) for all $j \in I$. Consider now the lifted inequality (6) for a given i such that $i \leq i_c$. The coefficients next to variables x_j are $\min\{w_j, C_i + 1\} = \min\{w_j, C_c + 1\} = w_j$ for all $j \neq i$, and the coefficient next to x_i is $\min\{w_c, C_i + 1\} = w_c$, so the inequalities (6) and (15) coincide. $\square$

The quality of the LP relaxation of the two models is computationally evaluated in Section 6.

5. Equivalence Between MCMKP and MPMKC

In this section we prove the following result:

Proposition 5.1. *Any feasible solution S of the MCMKP on an instance (I, p, w, C) corresponds to a feasible solution $S' = I \setminus S$ of the MPMKC on instance (I, p, w, C') where $C' = W - C$.*

Before showing this result, we will discuss the critical item of the MPMKC and a counterpart of the extended formulation for the MPMKC. As above, we assume that items are sorted in non-decreasing order according to their weights. Observe that the index of the critical item and the corresponding weight for the MPMKC are calculated differently.

Definition 5.2 (MPMKC Critical Weight and MPMKC Critical Item). Denote by

$$\bar{i}_c = \arg \max \left\{ i \in I \mid \sum_{j \geq i} w_j \geq C \right\}$$

the index of a *MPMKC-critical item*, i.e., the index of the first item that covers the knapsack, assuming all $i \geq \bar{i}_c$ will be taken as well. The *MPMKC-critical weight*, denoted by $\bar{w}_c$, is the weight of the critical item, i.e., $\bar{w}_c = w_{\bar{i}_c}$.

Proposition 5.3. *The weight of the smallest item taken into any feasible MPMKC solution is bounded from above by the critical weight $\bar{w}_c$, i.e.:*

$$S \text{ is feasible} \Rightarrow \min_{i \in S} w_i \leq \bar{w}_c.$$

Consequently, the capacity used by any minimal knapsack cover is bounded from above as:

$$w(S) \leq C + \bar{w}_c - 1.$$

Proof. Suppose S is a feasible MPMKC solution such that $\min_{i \in S} w_i > \bar{w}_c$. Hence, S contains only items i such that $i > \bar{i}_c$. However, by definition of the critical item, it follows that $\sum_{i: i > \bar{i}_c} w_i \leq C - 1$ which means that S is not a cover, a contradiction.

For the second part of the proposition, observe that capacity used by any feasible solution can be bounded from above as $C - 1 + w_{\max}$. However, knowing that removal of even the smallest item from this solution results in capacity $\leq C - 1$, we conclude that $w(S) \leq C - 1 + \bar{w}_c$. $\square$

We can now prove the following lemma:

Lemma 5.4. *We have that the index of a critical item i_c for the MCMKP on instance (I, p, w, C) and the index of a critical item $\bar{i}_c$ for the MPMKC on instance (I, p, w, C') are the same, i.e., it holds:*

$$\arg \min \left\{ i \in I \mid \sum_{j \leq i} w_j > C \right\} = \arg \max \left\{ i \in I \mid \sum_{j \geq i} w_j \geq W - C \right\}$$

Proof. Observe that $\sum_{j \leq i} w_j$ can be written as $W - \sum_{j > i} w_j$ so that i_c is the smallest index of an item i such that $\sum_{j > i} w_j < W - C$. Hence, for i_c it must hold: $\sum_{j \geq i_c} w_j \geq W - C$. Obviously, for any $i < i_c$, we have $\sum_{j \geq i} w_j \geq \sum_{j \geq i_c} w_j \geq W - C$. So, we conclude that i_c is the maximal index of an item that satisfies the latter property, which exactly corresponds to the definition of the index of a critical item $\bar{i}_c$ for the instance (I, p, w, C') for the MPMKC. $\square$

Extended Formulation for the MPMKC We now consider an (extended) MIP formulation for the MPMKC. The model utilizes binary variables y_i , for all $i \in I$, indicating the set of items

of a feasible MPMKC solution and an additional continuous variable $\tilde{z}$.

$$\max \sum_{i \in I} p_i y_i \quad (17)$$

$$\sum_{i \in I} w_i y_i \geq C \quad (18)$$

$$\sum_{i \in I} w_i y_i - \tilde{z} \leq C - 1 \quad (19)$$

$$\tilde{z} \leq w_i y_i + \bar{w}_c(1 - y_i) \quad \forall i \in I : i \leq \bar{i}_c \quad (20)$$

$$y_i \in \{0, 1\} \quad \forall i \in I \quad (21)$$

$$\tilde{z} \geq 0 \quad (22)$$

Constraint (18) ensures that the solution is a knapsack cover. Constraints (20) ensure that $\tilde{z}$ is bounded from above by the weight of the smallest item in the solution, by exploiting the property of Proposition 5.3. Finally, constraint (19) makes sure that the knapsack cover is minimal, i.e., whenever the smallest item is taken out of the solution, the weight of the remaining selected items becomes $\leq C - 1$.

Proof of Proposition 5.1. The linear transformation $y_i = 1 - x_i$, for all $i \in I$, allows to translate the extended formulation (F_e) for the MCMKP into the extended formulation (17)-(22) for the MPMKC and vice versa. We start with the model (F_e) given in Section 4.2, and apply this linear transformation. The model (F_e) translates into the following one:

$$P + \min \sum_{i \in I} (-p_i) y_i$$

$$\sum_{i \in I} w_i y_i \geq C' \quad (23)$$

$$\sum_{j \in I} w_j y_j - z \leq C' - 1 \quad (24)$$

$$z \leq w_c - (w_c - w_i) y_i \quad \forall i \in I : i \leq i_c \quad (25)$$

$$y_i \in \{0, 1\} \quad \forall i \in I$$

Observing that the minimization can be turned into maximization and, using the facts that $C' = C - W$ and $i_c = \bar{i}_c$ (see Lemma 5.4), it immediately follows that the transformed model and the model (17)-(22) are the same for the instance (I, p, w, C') , which concludes the proof. $\square$

Hence, in order to solve MPMKC on the instance (I, p, w, C) , it is sufficient to solve the complementary MCMKP on the instance $(I, p, w, W - C)$ and take the items left out of the optimal MCMKP solution. The above result together with Proposition 3.2 implies the following:

Corollary 5.5. *The problem MPMKC on an instance (I, p, w, C) is weakly NP-hard and there exists a DP algorithm that solves it in $O(n(W - C))$ time.*

6. Computational Study

The primary goal of this computational study is to compare the performances of the dynamic programming algorithm (DP) described in Section 3 with the MIP models discussed in Section 4. To the best of our knowledge, these models are the best formulations proposed so far in literature. The computational results presented in [7] allow us to conclude that both models, i.e., (F_l) and (F_e) , outperform in terms of computational performance the DP algorithm proposed in [7]. Recall that this latter DP algorithm runs in $O(n^2C)$ time. As a matter of fact, it already struggles to tackle instances with 100 items and it is effective only in solving small size instances (see [7] for more details). For this reason, we decided to exclude it from this computational study.

The experiments reported in this article have been performed on a computer with a 2.5 GHz Intel Xeon CPU processor and 64GB RAM, running a 64 bits Linux operating system. All the codes were compiled with g++ (version 4.8.4) using `-O3` as optimization option. We have tested the MIP models using `Cplex 12.6.3`, single-threaded (called for brevity `Cplex` in the following). All `Cplex` parameters are at their default values except the optimality tolerances which have been set to 0 in order to be able to compare the solutions of the models with the solutions of the DP algorithm. In all our tests we imposed a time limit of 1000 seconds and a memory limit of 3GB.

A second goal of this computational study is also to determine the size of the instances which can be solved to proven optimality within a short running time. In [7], a very diverse benchmark set of instances with up to 2000 items has been proposed. In this manuscript we consider the same classes of instances, but, in order to determine the limits of the proposed exact methods, we also introduce larger instances with up to 5000 items. In the following the test bed of instances is described in details.

6.1 Benchmark Instances

Nine classes of MCMKP instances have been randomly generated by defining weights and profits following the procedure for the classical KP described in [16] (u.r. stands for “uniformly random integer”). The instance generator is available at [20].

1. *Uncorrelated*: w_j u.r. in $[1, \bar{R}]$, p_j u.r. in $[1, \bar{R}]$.
2. *Weakly correlated*: w_j u.r. in $[1, \bar{R}]$, p_j u.r. in $[w_j - \bar{R}/10, w_j + \bar{R}/10]$ so that $p_j \geq 1$.
3. *Strongly correlated*: w_j u.r. in $[1, \bar{R}]$, $p_j = w_j + \bar{R}/10$.
4. *Inverse strongly correlated*: p_j u.r. in $[1, \bar{R}]$, $w_j = p_j + \bar{R}/10$.
5. *Almost strongly correlated*: w_j u.r. in $[1, \bar{R}]$, p_j u.r. in $[w_j + \bar{R}/10 - \bar{R}/500, w_j + \bar{R}/10 + \bar{R}/500]$.
6. *Subset-sum*: w_j u.r. in $[1, \bar{R}]$, $p_j = w_j$.
7. *Even-odd subset-sum*: w_j even value u.r. in $[1, \bar{R}]$, $p_j = w_j$, c odd.

8. *Even-odd strongly correlated*: w_j even value u.r. in $[1, \bar{R}]$, $p_j = w_j + \bar{R}/10$, c odd.
9. *Uncorrelated with similar weights*: w_j u.r. in $[100\bar{R}, 100\bar{R} + \bar{R}/10]$, p_j u.r. in $[1, \bar{R}]$.

For each instance class and value of $\bar{R} \in \{1000, 10000\}$, we generated 30 MCMKP instances by considering all combinations of (i) number of items $n \in \{10, 20, 30, 40, 50, 100, 500, 1000, 2000, 5000\}$; (ii) capacity $C \in \{\lfloor 25\%W \rfloor, \lfloor 50\%W \rfloor, \lfloor 75\%W \rfloor\}$ (and C increased by 1, if even, for classes 7 and 8); thus obtaining 540 MCMKP instances.

6.2 Warm Starting and Specialized Primal Heuristics

Thanks to extensive preliminary tests we observed that **Cplex** sometimes struggles to find good primal solutions for models (F_l) and (F_e) . Accordingly, we decided to warm start the MIP and implemented a specialized primal heuristic. The warm-start heuristic algorithm is called *greedy-comb* and it has been proposed in [7]. This approach consists of two basic steps:

- (i) examining the items according to a pre-specified order,
- (ii) adding the current item to the solution iff its weight does not exceed the current residual capacity.

Six different orderings are incorporated in this *greedy-comb* procedure, and the best solution found is returned. The procedure *greedy-comb* is very fast and produces high quality initial solutions, with gaps to the optimal solutions being between 1% and 8% on our benchmark set, see Table 2. The gaps are calculated as $(z_H - z^*)/z^* \cdot 100$, where z^* is the optimal solution value and z_H is the value of the heuristic solution.

Table 2: Average optimality gaps for the *greedy-comb* heuristic.

instance category								
1	2	3	4	5	6	7	8	9
7.67	1.06	0.98	1.49	0.99	2.18	2.20	0.92	3.01

In addition to this warm starting procedure, we apply a new and effective specialized primal heuristic that can be executed within the nodes of the branch-and-cut scheme of **Cplex**. Once an LP-solution $\bar{x}$ is calculated, we call *greedy-comb* by weighting the item scores using the values of $\bar{x}_i$, for all $i \in I$. Then the items are resorted according to the new score values, and *greedy-comb* is executed. This specialized primal heuristic produces effective speedups as it is shown in the next sections.

6.3 Quality of the LP Relaxation of the Models

Table 3 shows the gaps between the optimal solution value and the value of the LP relaxations of (F_l) and (F_e) . The table is structured according to the instance classes, i.e., each entry reports the average gap of the 60 instances of the considered classes. The gaps are calculated as $(z^* - z_R)/z^* \cdot 100$, where z^* is the optimal solution value and z_R is the optimal value of the LP relaxation. We observe that the gaps of both formulations are very similar, and that for classes 1, 2, 4, 5 and 9 the average gaps coincide. The instances of class 1 are characterized by the

largest average gaps (12.30%). Aside from this class and class 9 with an average gap of 5.98%, the gaps for all other classes are below 3%.

Table 3: Average percentage LP gaps for (F_l) and (F_e) .

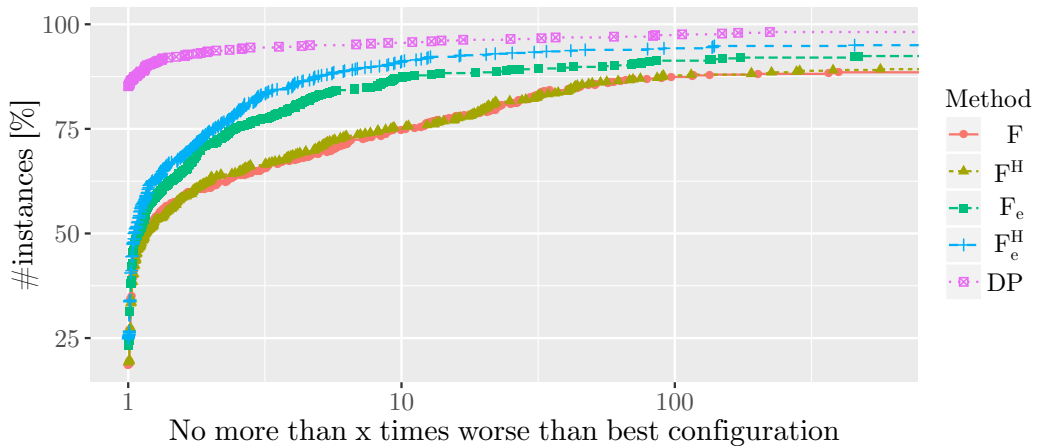
	instance class								
	1	2	3	4	5	6	7	8	9
(F_l)	12.30	2.87	2.47	2.27	2.41	2.63	2.64	2.56	5.98
(F_e)	12.30	2.87	2.51	2.27	2.41	2.65	2.66	2.59	5.98

6.4 Comparison Between the Proposed Exact Methods

The performance of the DP algorithm, the (F_l) and the (F_e) are compared. For the two MIP models additional configurations are considered, in which the basic models are enhanced by our warm-start procedure and specialized primal heuristic (denoted by $(F_l)^H$ and $(F_e)^H$). The computing time for the runs that have terminated due to time or memory limit is set to 1000 seconds.

In order to give a graphical representation of the relative performance of the proposed approaches, we report a performance profile in Figure 1. For each instance we compute a normalized time τ as the ratio of the computing time of the considered solution method over the minimum computing time for solving the instance to optimality. For each value of τ in the horizontal axis, the vertical axis reports the percentage of the instances for which the corresponding method spent at most τ times the computing time of the fastest method. This chart shows that DP outperforms the MIP models both in terms of speed and number of solved instances. In about 85% of the instances the DP algorithm is the fastest one and it manages to solve almost 99% of the instances to proven optimality. The second best approach is $(F_e)^H$ followed by (F_e) . The models $(F_l)^H$ and (F_l) have similar poor performances with respect to the DP algorithm. In the following, we will take a more detailed look at the results.

Figure 1: Performance profile of the different approaches.



An aggregated picture of our computational results is given in Table 4. The table reports the average runtime to solve to proven optimality all instances for a given number of items (n) (average values over 54 instances are reported in each line of the table). We can conclude

that instances with up to 100 items can be solved within a few seconds by all the proposed approaches. Starting with the instances having 500 or more items, the difficulty significantly increases, and the table computationally proves that the DP algorithm globally outperforms both MIP approaches for all classes of instances starting from $n = 30$ (with speedup factors ranging between 2 and 10).

As far as the effect of using the warm start and the specialized primal heuristic is concerned, the tables show that a sensible speed up can be obtained in most of the cases (excluding the trivial instances with less than 100 items). The effect is more marked up for (F_e) and 2000 items where the use of heuristics nearly halves the runtime. Interestingly, for (F_e) , the number of items does not seem to influence the runtime too much, as for 1000 items, the average runtime is actually slightly faster than for 500 items. On the other hand, as expected, for DP the runtime grows with the number of items.

Table 4: Performance comparison in term of average computing time by varying the number of items.

n	#	(F_l)	$(F_l)^H$	(F_e)	$(F_e)^H$	DP
10	54	0.00	0.00	0.00	0.00	0.00
20	54	0.01	0.02	0.02	0.02	0.02
30	54	0.07	0.11	0.12	0.13	0.04
40	54	0.17	0.35	0.39	0.41	0.07
50	54	0.20	0.35	0.21	0.21	0.10
100	54	1.22	13.16	13.23	5.83	0.42
500	54	164.15	178.57	171.25	145.85	10.72
1000	54	145.29	152.99	128.81	104.30	48.75
2000	54	333.35	317.16	292.83	159.67	76.86
5000	54	828.87	770.35	389.12	244.81	149.93

Table 5 reports the number of instances not solved to proven optimality within the given time limit (1000 seconds) and memory limit (3 GB). The structure of this table is the same as for Table 4, i.e., each line corresponds to a subset of 54 instances grouped by the number of items n . We see that both (F_l) and $(F_l)^H$ struggle to solve instances starting from $n = 500$. For the largest instances with $n = 5000$, 41 and 37 instances remain unsolved by (F_l) and $(F_l)^H$, respectively. Also (F_e) and $(F_e)^H$ struggle to solve instances starting from $n = 500$ but more instances could be solved, i.e, only 17 and 10 instances remain unsolved by (F_e) and $(F_e)^H$, respectively. A possible explanation is the density of the model (F_l) , that causes the memory limit to be reached more often than for the model (F_e) (which possesses an almost diagonal structure of the constraint matrix). We therefore conclude that model (F_e) outperforms model (F_l) in terms of the number of instances solved to proven optimality. Table 5 also demonstrates the beneficial effect of our heuristic algorithms: globally, $(F_l)^H$ solves 4 more instance than (F_l) and $(F_e)^H$ solves 14 more instances than (F_e) .

As far as the performance of the DP is concerned, the DP algorithm only starts struggling to solve instances with $n \geq 1000$ and only 10 out of 540 instances remain unsolved by this exact approach. As a comparison, model $(F_l)^H$ fails to prove optimality in 58 cases, whereas model (F_l) fails in 27 cases. This allows us to conclude that, with respect to the total number of instances solved to proven optimality, our new DP algorithm significantly outperforms both models $(F_l)^H$ and $(F_e)^H$.

Table 5: Instances unsolved within time limit and memory limit varying the number of items.

n	#	(F_l)	$(F_l)^H$	(F_e)	$(F_e)^H$	DP
10	54	0	0	0	0	0
20	54	0	0	0	0	0
30	54	0	0	0	0	0
40	54	0	0	0	0	0
50	54	0	0	0	0	0
100	54	0	0	0	0	0
500	54	8	9	9	7	0
1000	54	5	5	6	5	2
2000	54	8	7	9	5	3
5000	54	41	37	17	10	5

It is worth mentioning that all instances that remain unsolved for DP are of class 9, which has much larger weights compared to the other instance classes. This class of instances is specifically conceived to create instances which are hard for DP algorithms. In order to obtain a clearer picture concerning the performance of the proposed exact methods on the different classes of instances, we report Table 6. The table shows the average runtime needed to solve to proven optimality the instances grouped by class. Each column reports average values for 60 instances and in the rows the different exact methods are given. Runs that have terminated due to memory limit are counted as having a computing time of 1000 seconds as in the other tables.

Table 6: Performance comparison in term of average computing time grouping instances by class.

	instance category								
	1	2	3	4	5	6	7	8	9
#	60	60	60	60	60	60	60	60	60
(F_l)	115.47	97.85	113.94	287.77	213.64	164.23	149.37	98.06	86.94
$(F_l)^H$	119.10	80.40	99.42	293.30	211.08	157.00	166.98	73.05	90.56
(F_e)	17.25	11.14	223.68	270.88	111.09	56.06	23.10	154.65	28.53
$(F_e)^H$	17.82	8.73	92.56	240.61	99.01	45.54	13.52	46.49	30.82
DP	5.72	5.85	5.49	6.89	5.45	5.34	5.44	5.47	212.57

For both MIP approaches (regardless of using the heuristics or not), instance class 4 is the most difficult one, while at least for (F_e) (and $(F_e)^H$), instances of class 1 and 2 are rather easy. For all classes except class 9, the DP algorithm clearly outperforms the MIP approaches. As previously mentioned, the instances of class 9 have much larger coefficients which leads to a higher value of C and accordingly the performance of the DP algorithm deteriorates. For the instances of class 9, (F_e) tends to be the best approach. Finally, we can conclude that (F_e) outperforms (F_l) on all classes of instances.

In Tables 7 ($\bar{R} = 1000$) and 8 ($\bar{R} = 10000$) we show a detailed overview of the total runtime for instances with $n \geq 100$ items. We report the runtime for $(F_l)^H$, $(F_e)^H$ and DP . Observe that the instances that were not solved to optimality by DP, were solved by $(F_e)^H$, and vice versa, so that we are able to report optimal solution values for the whole benchmark set. These values are shown in columns z^* .

Summary This computational study demonstrates that the DP algorithm outperforms the MIP-based approaches for 8 out of 9 instance classes. The only exception are the instances of class 9. Recall that class 9 is built with about 100 times larger weights than the remaining classes.

Table 7: Runtime in seconds and best solution value for each instance with $\bar{R} = 1000$.

cat.	$n \setminus C$	$(F_l)^H$			$(F_e)^H$			DP			z^*		
		25%	50%	75%	25%	50%	75%	25%	50%	75%	25%	50%	75%
1	100	0.24	0.38	0.46	0.40	0.11	0.33	0.00	0.00	0.00	2396	9418	20420
	500	1.27	1.15	4.98	0.85	0.25	1.45	0.04	0.08	0.12	11034	43945	95648
	1000	3.14	13.10	0.90	1.56	1.13	0.27	0.16	0.33	0.48	22225	90721	199140
	2000	50.59	49.23	126.22	7.75	6.47	7.65	0.64	1.29	1.94	47146	189380	417695
	5000	76.12	359.79	ML	20.69	17.28	17.15	3.97	8.91	11.88	115800	470676	1058114
2	100	0.36	0.36	0.73	0.22	0.16	0.52	0.00	0.00	0.00	10246	22062	34855
	500	0.63	4.62	5.75	0.41	0.69	2.44	0.04	0.09	0.13	49973	107440	169917
	1000	8.00	4.94	1.22	1.93	1.26	0.30	0.17	0.33	0.50	100823	216924	342464
	2000	20.99	47.40	25.50	3.99	5.92	6.01	0.67	1.32	2.03	206505	444553	701755
	5000	68.01	45.38	ML	13.93	6.76	17.99	4.10	8.29	12.52	524047	1122836	1768368
3	100	0.05	0.13	0.41	0.15	0.04	0.02	0.00	0.00	0.00	13991	28182	42873
	500	0.58	7.73	1.09	0.12	0.44	0.32	0.03	0.07	0.11	68609	138518	210727
	1000	1.18	1.56	6.49	0.51	1.10	1.77	0.14	0.29	0.44	133599	269798	410797
	2000	3.22	6.32	5.40	1.58	3.07	1.52	0.59	1.18	1.78	269059	543419	827379
	5000	32.86	ML	ML	18.97	40.46	12.24	3.79	8.59	12.40	683746	1379993	2100739
4	100	0.04	0.37	0.19	0.05	0.58	0.10	0.00	0.00	0.00	10117	22852	35951
	500	TL	3.20	TL	TL	8.73	TL	0.05	0.10	0.15	51203	114165	179505
	1000	10.02	28.87	6.86	2.12	12.81	1.86	0.19	0.38	0.57	99064	221845	349770
	2000	5.82	TL	140.83	2.13	230.83	7.52	0.77	1.54	2.31	200321	448155	706438
	5000	363.73	ML	ML	TL	19.96	41.41	4.88	10.78	15.96	512400	1142721	1799085
5	100	0.75	0.11	0.37	0.10	0.01	0.18	0.00	0.00	0.00	13744	27599	42046
	500	TL	1.16	1.66	467.47	0.26	0.32	0.03	0.07	0.11	66739	134695	205247
	1000	1.21	3.90	TL	0.39	1.54	TL	0.14	0.29	0.44	133601	269804	411130
	2000	64.65	12.77	209.30	10.76	11.33	6.06	0.60	1.21	1.81	272438	549814	836905
	5000	TL	478.32	ML	11.80	29.52	17.08	3.83	8.61	12.23	689237	1390675	2116720
6	100	0.07	0.18	0.32	0.04	0.08	0.17	0.00	0.00	0.00	12067	24538	36953
	500	2.28	2.51	19.48	0.29	0.66	3.41	0.03	0.07	0.11	61295	122941	184592
	1000	27.39	116.15	164.23	3.38	4.16	22.42	0.14	0.29	0.44	119912	240120	360361
	2000	24.64	826.12	627.04	6.58	2.61	10.32	0.59	1.18	1.78	242370	485031	727723
	5000	909.99	ML	ML	44.18	40.45	155.50	3.78	7.61	12.26	617052	1234392	1851776
7	100	0.07	0.15	0.24	0.04	0.05	0.10	0.00	0.00	0.00	12080	24564	36994
	500	1.25	0.85	9.14	0.56	2.13	0.23	0.03	0.07	0.11	61358	123068	184782
	1000	49.13	6.35	97.29	0.98	1.42	3.64	0.14	0.29	0.44	120034	240368	360732
	2000	897.75	TL	250.01	14.79	2.82	6.24	0.59	1.18	1.80	242616	485522	728458
	5000	TL	ML	ML	44.06	33.20	59.05	3.81	7.56	11.57	617678	1235644	1853652
8	100	0.35	0.01	0.02	0.02	0.00	0.01	0.00	0.00	0.00	14004	28210	42916
	500	0.59	0.29	1.04	0.18	0.19	0.34	0.03	0.07	0.11	68672	138644	210916
	1000	1.08	1.58	1.69	0.45	0.55	0.45	0.14	0.29	0.44	133722	270046	411168
	2000	2.15	4.57	5.87	0.57	1.53	1.91	0.59	1.18	1.79	269304	543910	828214
	5000	15.71	47.92	ML	3.18	25.67	23.57	3.79	7.71	12.59	684372	1381344	2102816
9	100	0.00	0.00	0.02	0.00	0.00	0.01	0.29	0.65	0.98	2921	11548	26568
	500	0.03	0.09	0.17	0.02	0.03	0.05	8.12	16.33	25.30	14563	58423	132701
	1000	0.17	0.44	0.66	0.07	0.12	0.18	33.46	66.06	98.96	29462	116287	265579
	2000	0.82	TL	TL	0.24	695.56	TL	130.85	264.36	396.21	61180	240024	547607
	5000	717.22	17.61	ML	23.30	2.73	4.26	826.20	TL	TL	159035	621589	1397221

Table 8: Runtime in seconds and best solution value for each instance with $\bar{R} = 10000$.

cat.	$n \setminus C$	$(F_l)^H$			$(F_e)^H$			DP			z^*		
		25%	50%	75%	25%	50%	75%	25%	50%	75%	25%	50%	75%
1	100	3.19	20.29	4.68	1.35	20.22	2.13	0.02	0.04	0.06	21811	80166	179311
	500	53.62	33.51	209.00	4.86	12.02	57.95	0.46	0.89	1.32	114523	459156	1039992
	1000	43.44	834.24	49.45	21.51	181.38	10.50	1.71	3.38	5.16	224150	941497	2117198
	2000	330.56	807.16	423.48	60.51	178.36	64.81	6.95	13.34	21.25	483979	1925732	4295218
	5000	628.31	ML	ML	120.47	64.96	176.26	44.68	86.53	127.29	1179665	4686830	10507765
2	100	0.10	0.13	0.32	0.08	0.07	0.24	0.02	0.05	0.07	85229	185013	292897
	500	3.10	8.25	2.47	0.97	4.56	5.31	0.49	0.94	1.37	498746	1073343	1698472
	1000	10.71	57.56	33.52	3.55	7.58	19.88	1.81	3.72	5.56	1028577	2201879	3468177
	2000	145.06	144.01	685.58	18.73	36.83	90.64	7.15	14.44	21.59	2071531	4430501	6976442
	5000	492.25	ML	ML	24.54	134.98	112.60	44.24	89.06	130.24	5263091	11251628	17711417
3	100	0.17	53.55	296.85	0.33	80.08	34.27	0.01	0.02	0.04	118252	238588	363929
	500	2.80	36.61	2.67	0.73	237.60	4.20	0.36	0.71	1.13	649309	1310618	2000927
	1000	67.69	294.28	6.28	116.11	8.90	4.05	1.49	3.20	4.59	1351149	2726298	4156447
	2000	92.90	13.36	10.70	975.22	TL	3.46	6.13	13.42	19.62	2747609	5545219	8442829
	5000	ML	ML	ML	TL	TL	TL	40.73	82.14	126.33	6926796	13981593	21278389
4	100	0.12	6.73	20.12	0.08	5.43	12.39	0.03	0.05	0.08	79817	186007	297960
	500	TL	TL	TL	TL	TL	TL	0.51	1.04	1.57	476256	1068850	1693224
	1000	TL	TL	TL	TL	TL	TL	1.96	4.10	6.36	1010792	2247788	3541015
	2000	TL	TL	TL	TL	89.59	TL	7.83	16.77	24.67	2066109	4591558	7222366
	5000	ML	ML	ML	TL	TL	TL	51.66	104.00	155.23	5230252	11605856	18236923
5	100	0.17	5.31	1.80	138.17	2.47	1.79	0.01	0.03	0.05	114770	230670	352572
	500	TL	TL	TL	TL	TL	9.43	0.38	0.77	1.18	664526	1343087	2046816
	1000	47.03	TL	7.19	40.64	TL	4.01	1.53	3.05	4.75	1349293	2725604	4150205
	2000	343.94	220.77	247.50	95.10	28.10	7.73	6.53	13.19	19.48	2733192	5519666	8401427
	5000	TL	ML	ML	24.60	TL	17.32	40.33	82.60	124.00	6909253	13944843	21218933
6	100	0.08	0.26	0.04	0.06	0.14	0.05	0.01	0.02	0.04	101130	204809	308231
	500	4.78	32.60	36.67	1.13	3.75	12.74	0.36	0.72	1.08	580840	1163941	1747413
	1000	37.97	125.58	169.78	4.18	7.62	48.17	1.49	3.03	4.49	1213512	2429569	3645954
	2000	302.56	439.40	539.30	15.96	14.52	14.71	6.06	12.65	19.84	2474759	4952346	7430298
	5000	ML	ML	ML	909.91	404.44	TL	40.36	80.72	121.34	6248810	12500605	18752821
7	100	0.08	0.27	0.05	0.06	0.12	0.05	0.01	0.02	0.04	101156	204848	308280
	500	4.31	10.81	28.58	0.74	3.10	10.70	0.35	0.71	1.07	580892	1164068	1747602
	1000	42.64	200.22	115.74	3.85	13.58	35.39	1.49	3.16	4.88	1213636	2429818	3646326
	2000	168.13	279.62	845.65	13.94	90.86	43.43	6.64	12.56	19.33	2475004	4952838	7431034
	5000	ML	ML	ML	14.07	223.02	188.44	41.21	83.21	124.01	6249436	12501856	18754698
8	100	0.16	0.53	289.00	0.62	0.24	10.76	0.01	0.02	0.04	118262	237710	363974
	500	0.95	16.79	82.33	0.29	8.52	4.98	0.35	0.71	1.07	649372	1310744	2001116
	1000	39.14	280.84	226.95	1.52	5.70	24.75	1.49	2.98	4.59	1351272	2726546	4156818
	2000	5.90	288.46	28.49	5.09	TL	645.67	6.13	13.07	20.14	2747854	5545710	8443564
	5000	ML	25.03	ML	TL	5.87	7.71	40.53	87.43	120.95	6927422	13982844	21280266
9	100	0.00	0.00	0.01	0.00	0.00	0.01	3.31	6.73	10.06	18762	84731	207429
	500	0.05	0.11	1.12	0.02	0.04	0.67	83.89	176.32	247.69	145911	596495	1326550
	1000	8.79	4.10	0.84	0.56	2.03	0.23	346.45	TL	TL	302563	1216230	2428114
	2000	350.56	2.31	3.78	71.66	0.57	0.87	TL	TL	TL	628397	2428114	2428114
	5000	306.25	14.59	ML	36.89	3.18	6.06	TL	TL	TL	1577597	2428114	2428114

Since C is defined as a percentage of the sum of all item weights, and the time complexity of DP is $O(nC)$, this easily explains the relatively poor performance of DP on this instance class. On the contrary, for the same class, $(F_e)^H$ seems to be the best performing method. For the DP algorithm, Tables 7 and 8 confirm that the runtime grows with n and with C . For all approaches, instances with $\bar{R} = 1000$ tend to be easier than the ones with $\bar{R} = 10000$, which can be explained by smaller item weights in the former case. No clear picture emerges instead regarding a possible correlation between the value of the capacity C and the performance of the MIP-based approaches. For example, for $1000 - 4 - 2000$ ($\bar{R} - cat - n$), $(F_e)^H$ takes 239 seconds for $C = 50\%$, against 2 seconds for $C = 25\%$ and 8 seconds for $C = 75\%$, whereas for $1000 - 4 - 500$ the results are reversed, with both $C = 25\%$ and $C = 75\%$ remaining unsolved within the time limit, and $C = 50\%$ being solved in 8 seconds only. The most difficult instance class for the MIP-based approaches is class 4, with $\bar{R} = 10000$, for which very few instances can be solved to optimality. On the other hand, the DP algorithm requires at most 155 seconds for these instances.

7. Conclusions and Future Work

Problems which impose inclusion-wise maximality of a feasible solution while at the same time minimizing a non-negative cost function, have recently received much attention from the Theoretical Computer Science community. In particular, the so-called Lazy Bureaucrat Problem has been studied from approximation and complexity points of view (see e.g., [11] and the references therein).

In this article we studied two versions of this problem denoted by the Minimum-Cost Maximal Knapsack Packing (MCMKP) and the Maximum-Profit Minimal Knapsack Cover (MPMKC). These are two problems closely related to the knapsack problem, with very important applications in scheduling. In this manuscript we proposed an exact algorithm based on dynamic programming and two effective MIP formulations. Our methods allowed us to solve to proven optimality instances with up to 5 000 items in short computing time.

Concerning future work, it is worth mentioning that many related “lazy” problems that have recently been introduced in the Computer Science literature (see, e.g., [6, 9] for the lazy bin packing problem, [5] for the minimum maximal independent set problem, [4] for the max-min vertex cover problem, [2] for the maximum cardinality minimal dominating set problem, or [12] for the lazy matroid problem) remain quite unexplored from the perspective of exact algorithms. With this article we hope to stimulate the research on this new and challenging family of problems which can give rise to new solution properties and exact methods. It could be interesting to study other variants of the classical knapsack problem in this “lazy” setting, such as quadratic knapsack and covering problems [10, 19] or the multiple-choice knapsack problem [18]. Finally, the investigation of speed-up techniques for dynamic programming algorithms such as state-space reduction (see, e.g., [14]) could also be interesting to further improve the performance of the proposed algorithm.

Acknowledgments

The authors would like to thank the two anonymous referees for their valuable comments which helped to improve the quality of the paper. The research of M. Sinnl was supported by the Austrian Research Fund (FWF, Project P 26755-N19).

References

- [1] Esther M. Arkin, Michael A. Bender, Joseph S.B. Mitchell, and Steven S. Skiena. The lazy bureaucrat scheduling problem. *Information and Computation*, 184(1):129–146, 2003.
- [2] Cristina Bazgan, Ljiljana Brankovic, Katrin Casel, Henning Fernau, Klaus Jansen, Kim-Manuel Klein, Michael Lampis, Mathieu Liedloff, Jérôme Monnot, and Vangelis Th Paschos. Upper domination: Complexity and approximation. In Veli Mäkinen, Simon J. Puglisi, and Leena Salmela, editors, *International Workshop on Combinatorial Algorithms*, volume 9843 of *Lecture Notes in Computer Science*, pages 241–252. Springer, 2016.
- [3] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [4] Nicolas Boria, Federico Della Croce, and Vangelis Th Paschos. On the max min vertex cover problem. *Discrete Applied Mathematics*, 196:62–71, 2015.
- [5] Nicolas Bourgeois, Federico Della Croce, Bruno Escoffier, and V Th Paschos. Fast algorithms for min independent dominating set. *Discrete Applied Mathematics*, 161(4):558–572, 2013.
- [6] Joan Boyar, Leah Epstein, Lene M. Favrholdt, Jens S. Kohrt, Kim S. Larsen, Morten M. Pedersen, and Sanne Wøhlk. The maximum resource bin packing problem. *Theoretical Computer Science*, 362(1–3):127–139, 2006.
- [7] Fabio Furini, Ivana Ljubić, and Markus Sinnl. ILP and CP Formulations for the Lazy Bureaucrat Problem. In Lauren Michel, editor, *International Conference on AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, volume 9075 of *Lecture Notes in Computer Science*, pages 255–270. Springer, 2015.
- [8] Ling Gai and Guochuan Zhang. On lazy bureaucrat scheduling with common deadlines. *Journal of Combinatorial Optimization*, 15(2):191–199, 2008.
- [9] Ling Gai and Guochuan Zhang. Hardness of lazy packing and covering. *Operations Research Letters*, 37(2):89 – 92, 2009.
- [10] Fred Glover. Advanced greedy algorithms and surrogate constraint methods for linear and quadratic knapsack and covering problems. *European Journal of Operational Research*, 230(2):212 – 225, 2013.
- [11] L. Gourvès, J. Monnot, and Aris T. Pagourtzis. The lazy bureaucrat problem with common arrivals and deadlines: Approximation and mechanism design. In Leszek Gasieniec and Frank Wolter, editors, *Fundamentals of Computation Theory*, volume 8070 of *Lecture Notes in Computer Science*, pages 171–182. Springer Berlin Heidelberg, 2013.

- [12] Laurent Gourvès, Jérôme Monnot, and Aris T Pagourtzis. The lazy matroid problem. In Josep Diaz, Ivan Lanese, and Davide Sangiorgi, editors, *IFIP International Conference on Theoretical Computer Science*, volume 8705 of *Lecture Notes in Computer Science*, pages 66–77. Springer, 2014.
- [13] Clint Hepner and Cliff Stein. Minimizing makespan for the lazy bureaucrat problem. In Martti Penttonen and Erik Meineche Schmidt, editors, *Scandinavian Workshop on Algorithm Theory*, volume 2368 of *Lecture Notes in Computer Science*, pages 40–50. Springer, 2002.
- [14] H. Kellerer, U. Pferschy, and D. Pisinger. *Knapsack Problems*. Springer, Berlin, 2004.
- [15] S. Martello and P. Toth. *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons, Chichester, 1990.
- [16] S. Martello, D. Pisinger, and P. Toth. Dynamic programming and strong bounds for the 0-1 knapsack problem. *Management Science*, 45:414–424, 1999.
- [17] Silvano Martello, David Pisinger, and Paolo Toth. New trends in exact algorithms for the 0-1 knapsack problem. *European Journal of Operational Research*, 123(2):325 – 332, 2000.
- [18] David Pisinger. A minimal algorithm for the multiple-choice knapsack problem. *European Journal of Operational Research*, 83(2):394–410, 1995.
- [19] David Pisinger. The quadratic knapsack problem – a survey. *Discrete Applied Mathematics*, 155(5):623–648, 2007.
- [20] David Pisinger. David Pisinger’s optimization codes, 2014. <http://www.diku.dk/~pisinger/codes.html>.
- [21] H Paul Williams. Fourier’s method of linear programming and its dual. *The American mathematical monthly*, 93(9):681–695, 1986.